Project Title

Full-Text Search Engine on a Suffix-Array Index

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
P Neeharika (2520090233)
G Shivani (2520090046)

Team Number: 25

Section: 10

Under the guidance of

<<Mrs. Ch. Anitha>>



<<Asst. Professor, CS&IT>>

Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING / COMPUTER SCIENCE AND INFORMATION TECHNOLOGY/ARTIFICIAL INTELLIGENCE & DATA SCIENCE



DECLARATION

The Mini Project Report entitled "**Full-Text Search Engine on a Suffix-Array Index"** is a record of Bonafide work of **P Neeharika – 2520090233, G Shivani – 2520090046,** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering/Computer Science and Information Technology/ Artifical Intelligence & data Science to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

P Neeharika – 2520090233

G Shivani – 2520090046

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING / COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project based report entitled "**Full-Text Search Engine on a Suffix-Array Index"** is a bonafide work done and submitted by **P Neeharika – 2520090233, G Shivani– 2520090046,** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering/ Computer Science and Information Technology/Artificial Intelligence & Data Science, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, Mrs. Ch. Anitha, Asst. Professor, CSIT which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P.Venkateshwar Rao (CSE)/Dr. Khaja Shareef (CSIT)/ Dr. Sreenath Itikela (AI&DS)**, Head of the *Department for Computer Science Engineering/ Department for Computer Science and Information Technology/Department of Artificial Intelligence and Data Science* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

P Neeharika – 2520090233

G Shivani– 2520090046

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8
4.	Problem Statement and Objectives	10
5.	Hardware and Software requirements	11
6.	Implementation	12
7.	Results & Discussion	16
8.	Conclusion & Future Scope	19
9.	References	20

1.ABSTRACT

The **Full-Text Search Engine on a Suffix-Array Index** is a Java-based search system designed to provide efficient searching across multiple text documents. The project addresses the problem of repeatedly scanning large amounts of text for every search query by creating an indexed representation of the combined document content. The system constructs a **suffix array** using a practical **O(n log² n) prefix-doubling approach**, allowing suffixes of the text to be arranged in lexicographical order and searched using binary search. An **LCP (Longest Common Prefix) array** is also generated using **Kasai's algorithm** in O(n) time to identify common prefixes between neighbouring suffixes and support efficient searching. Users can upload their own .txt documents, build the index, and perform substring, prefix, and exact-phrase searches through a graphical interface. Matching documents are identified and ranked based on the number of occurrences of the searched query. The system demonstrates how suffix-array indexing can improve full-text search by preprocessing the text once and reusing the resulting index for multiple queries. The project provides a practical implementation of string-processing concepts and demonstrates the application of **suffix arrays, LCP arrays, binary search, and string matching** in a real-world search system.

Team Name: Full-Text Search Engine on a Suffix-Array Index

Team Members:

1. [P Neeharika, 2520090233]
2. [G Shivani, 2520090046]

2. INTRODUCTION

Nowadays, we have a large amount of information stored in text files, so finding specific information quickly is important. Searching through every document again and again can take more time, especially when the number of documents becomes large. To solve this problem, our project uses a **suffix array** to organize the text and make searching easier.

The **Full-Text Search Engine on a Suffix-Array Index** is a Java-based project that allows users to upload their own text files and search for specific words or phrases. The system first combines the uploaded documents and creates a **suffix array** for the text. The suffix array is built using the **prefix-doubling method**, and an **LCP (Longest Common Prefix) array** is created using **Kasai's algorithm**.

After building the index, users can enter a search query and find matching text in the uploaded documents. The system also shows the documents containing the query and ranks them based on how many times the query appears.

This project helps us understand how **suffix arrays, LCP arrays, binary search, and string matching** can be used to build a practical search system. The following sections explain the algorithms used, system design, implementation, results, and conclusion of the project.

3. LITERATURE SURVEY

1. Apache Lucene

Apache Lucene is a Java-based search library used to add search functionality to applications. It helps developers index documents and search their content efficiently. It supports text searching and provides methods to retrieve relevant documents. Our project is similar because it also uses indexing techniques to search text documents, but we implement a suffix array instead of depending on an existing search library.

2. Elasticsearch

Elasticsearch is a search engine that helps users search and analyze large amounts of data. It uses an inverted index to organize text and retrieve matching documents. It also supports phrase searching and ranking results based on relevance. Our project has a similar purpose, but it focuses on implementing suffix-array indexing and binary search to understand the algorithms behind text searching.

3. Apache Solr

Apache Solr is an open-source search platform built on Apache Lucene. It allows users to index documents and search their contents using different query types. It supports phrase searching, filtering, and ranking search results. Our project follows a similar approach to document searching, but uses a simpler Java application to demonstrate suffix arrays and LCP arrays.

4. SQLite Full-Text Search (FTS5)

SQLite FTS5 is a full-text search feature used to search text stored in databases. It supports searching for words, phrases, and prefixes, and can rank matching results. It uses an indexing technique to make searching more efficient. Our project provides similar search features for uploaded text files but focuses on suffix arrays and the LCP array rather than a database-based inverted index.

5. Whoosh Search Library

Whoosh is a search library that allows developers to create custom search applications. It supports indexing documents, searching text fields, and retrieving matching results. Developers can use it to add search functionality to applications such as websites and blogs. Our project has a similar goal, but we build the main searching algorithms ourselves in Java to demonstrate the working of suffix-array indexing.

6. GNU Grep

GNU Grep is a command-line tool used to search for specific patterns in files. It displays lines that contain the given pattern and supports different pattern-matching options. It is useful for finding information in source code and text files. Our project also searches text files for user-entered queries, but provides a graphical interface and uses a suffix array to organize text for searching.

7. ripgrep

ripgrep is a fast text-search tool that can search files and directories recursively. It supports regular expressions and automatically skips certain files that are usually not needed for searching. It is widely useful for finding text in large software projects. Our project is similar in its purpose of locating text, but focuses on searching uploaded documents using a suffix array and displaying ranked document results.

8. Meilisearch

Meilisearch is a search engine designed to provide fast and user-friendly search results. It supports full-text search, typo tolerance, and relevance-based ranking. It is used in applications such as websites, product catalogs, and knowledge bases. Our project also aims to provide quick search results and rank matching documents, but focuses on demonstrating suffix arrays, binary search, and the LCP array as core data structures and algorithms.

4. Problem Statement and Objectives

Problem Statement: Traditional text searching becomes inefficient when repeatedly searching through large collections of documents.

Problem Description: Searching each document from the beginning for every query takes more time as the amount of text increases. A faster indexing-based method is needed to search multiple text documents efficiently.

Objectives:

· To develop a full-text search engine for multiple text documents.

· To allow users to upload and search their own .txt files.

· To construct a **suffix array** for efficient text indexing.

· To construct the **LCP array using Kasai's algorithm**.

· To use **binary search** for faster query searching.

· To identify matching documents and rank them based on the number of query occurrences.

· To provide a simple graphical interface for uploading documents and displaying search results.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

· Processor: Intel Core i3 or equivalent
· RAM: 4 GB or higher
· Storage: 500 MB or more free space

Software Requirements:

· Operating System: Windows / Linux
· Programming Language: Java (JDK 17 or above)
· IDE: Visual Studio Code / Eclipse / IntelliJ IDEA

6. IMPLEMENTATION

Module 1 – Document Upload

```
public void uploadDocuments() {
        JFileChooser chooser = new JFileChooser();
        chooser.setMultiSelectionEnabled(true);

        if (chooser.showOpenDialog(frame) ==
JFileChooser.APPROVE_OPTION) {
                File[] files = chooser.getSelectedFiles();
                documents = new Documents[files.length];

                for (int i = 0; i < files.length; i++) {
                        String content = Files.readString(files[i].toPath());
                        documents[i] = new Documents(files[i].getName(),
content);
                }
        }
}
```

Module 2 – Suffix Array Construction

```
private int[] buildSuffixArray(String text) {
    int n = text.length();
    int[] sa = new int[n];
    int[] rank = new int[n];
```

```
        for (int i = 0; i < n; i++) {
            sa[i] = i;
            rank[i] = text.charAt(i);
        }

        for (int length = 1; length < n; length *= 2) {
            mergeSort(sa, rank, length, 0, n - 1);

            int[] newRank = new int[n];
            newRank[sa[0]] = 0;

            for (int i = 1; i < n; i++) {
                newRank[sa[i]] = newRank[sa[i - 1]] + 1;
            }

            rank = newRank;
        }

        return sa;
    }
```

Module 3 – LCP Array using Kasai's Algorithm

```
    private void buildLCP() {
        int n = text.length();
        lcp = new int[n];
        int[] rank = new int[n];

        for (int i = 0; i < n; i++)
            rank[suffixArray[i]] = i;
```

```
        int k = 0;

        for (int i = 0; i < n; i++) {
            if (rank[i] == n - 1) {
                k = 0;
                continue;
            }

            int j = suffixArray[rank[i] + 1];

            while (i + k < n && j + k < n &&
                    text.charAt(i + k) == text.charAt(j + k)) {
                k++;
            }

            lcp[rank[i]] = k;

            if (k > 0)
                k--;
        }
    }
```

Module 4 – Search and Ranking

```
    public int[] search(String query) {
        int first = findFirst(query);

        if (first == -1)
            return new int[0];

        int last = findLast(query);
```

```
        int[] positions = new int[last - first + 1];

        for (int i = 0; i < positions.length; i++) {
            positions[i] = suffixArray[first + i];
        }

        return positions;
    }

    public int countMatches(String content, String query) {
        int count = 0;

        for (int i = 0; i <= content.length() - query.length(); i++) {
            boolean match = true;

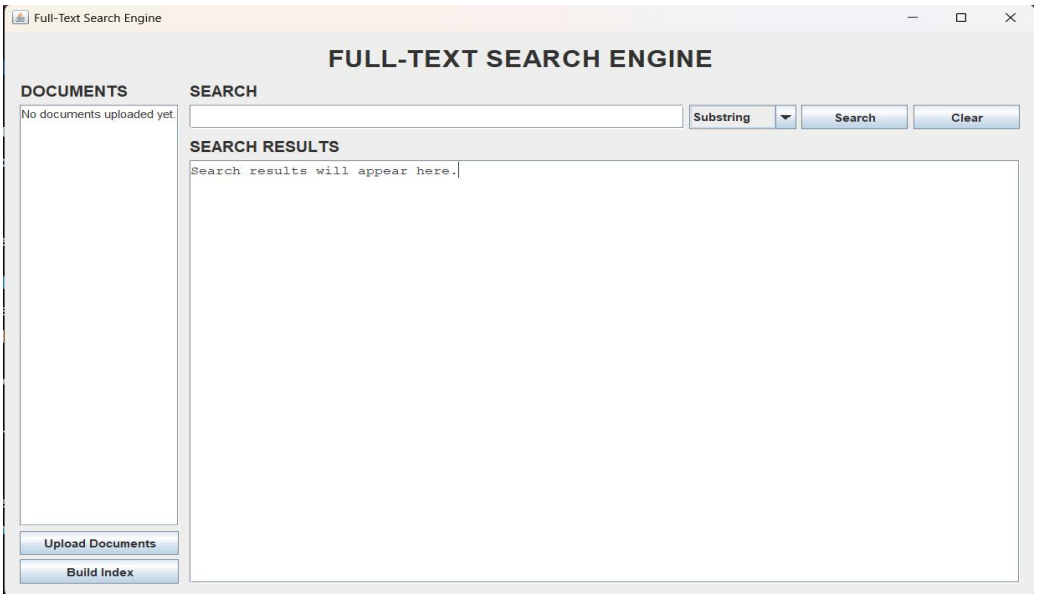
            for (int j = 0; j < query.length(); j++) {
                if (content.charAt(i + j) != query.charAt(j)) {
                    match = false;
                    break;
                }
            }

            if (match)
                count++;
        }

        return count;
    }
```

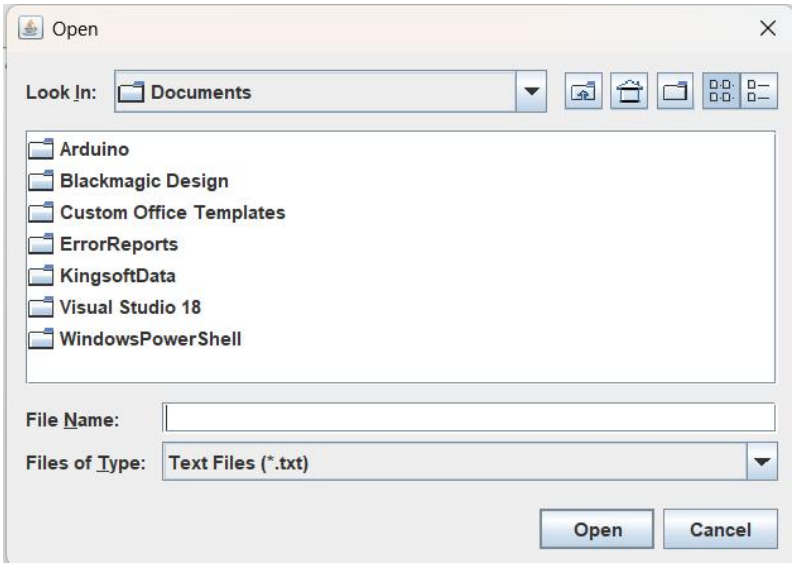
7. RESULTS & DISCUSSION

1. Main Application Screen



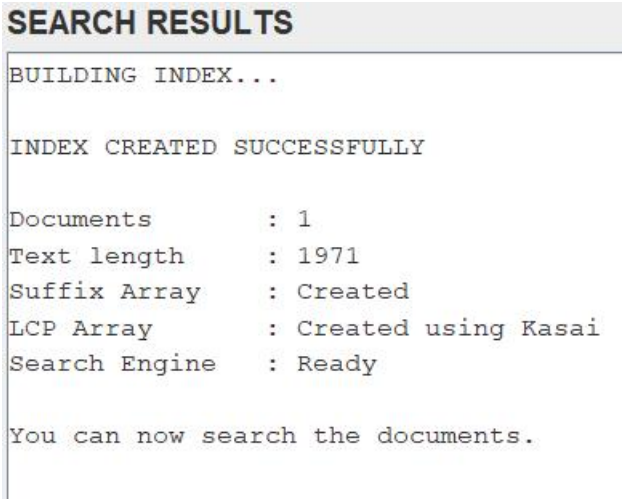
It provides the basic controls required to perform the full-text search.

2. Documents Uploaded



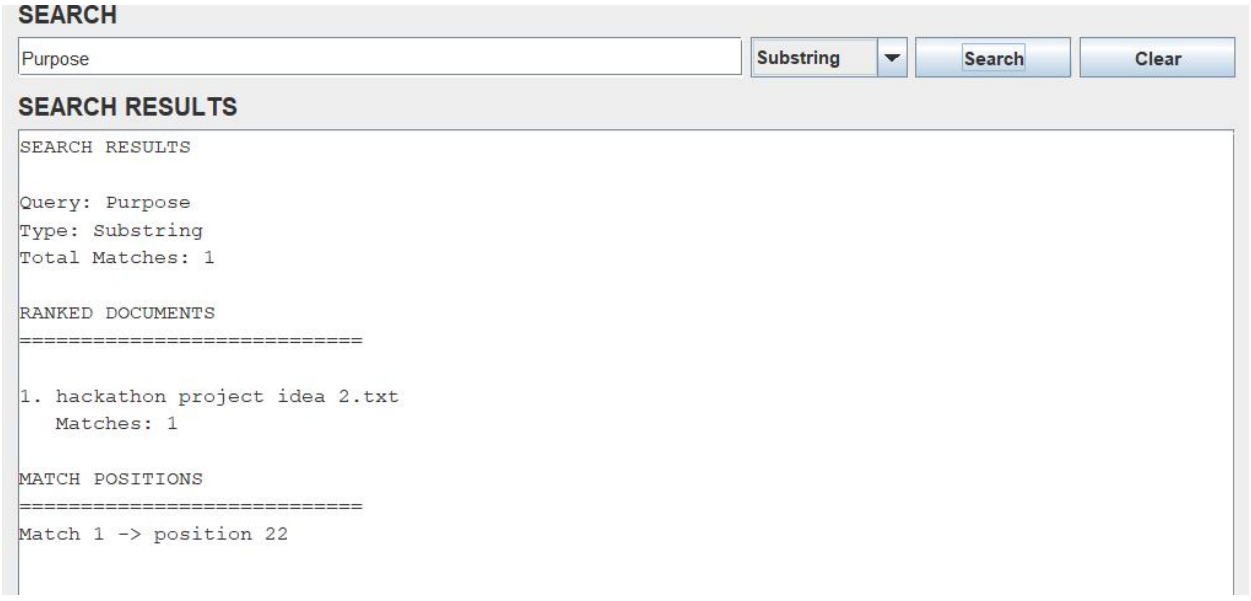
Shows the .txt documents selected and loaded into the application.
The uploaded documents are displayed before building the search index.

3. Index Built Successfully



Shows the suffix array and LCP index being created after the documents are uploaded.
This prepares the text for faster searching using the implemented algorithms.

4. **Search Results**



Shows the results obtained after entering a search query.
Matching documents and the number/positions of occurrences are displayed to the user.

5. Ranked Search Results

```
RANKED DOCUMENTS
============================

1. hackathon project idea 2.txt
   Matches: 1
```

Shows the matching documents arranged according to the number of times the query occurs.
Documents with more occurrences are given a higher ranking in the search results.

8. CONCLUSION & FUTURE SCOPE

The **Full-Text Search Engine on a Suffix-Array Index** successfully provides a simple way to search through multiple text documents. The project allows users to upload .txt files, build a suffix-array index, and search for words or phrases efficiently. The implementation of the **suffix array, LCP array using Kasai's algorithm, binary search, and document ranking** helped achieve the main objectives of the project. Overall, the project demonstrates how string-processing algorithms and data structures can be used to build a practical full-text search system.

In the future, the project can be improved by supporting more document formats such as PDF and DOCX. The search system can also be enhanced to handle larger collections of documents and provide more accurate ranking of search results. Features such as fuzzy search, typo correction, and advanced search options can be added. The application can also be developed as a web-based search engine for easier access and use.

9. REFERENCES

Apache Lucene – Java-based full-text search library.
https://lucene.apache.org/core/9_12_0/index.html

Elasticsearch – Documentation on full-text searching and document indexing.
https://www.elastic.co/docs/solutions/search/full-text

Apache Solr – Search platform supporting document indexing and search.
https://solr.apache.org/guide/solr/latest/index.html

SQLite FTS5 – Documentation on full-text search, phrase queries, and prefix searches.
https://www.sqlite.org/fts5.html

Suffix Array Construction – An Elegant Algorithm for the Construction of Suffix Arrays – Research article explaining suffix arrays and their construction.
https://pmc.ncbi.nlm.nih.gov/articles/PMC4101535/

Suffix Array Construction – Encyclopedia of Algorithms – Reference explaining suffix arrays and their use in text indexing.
https://doi.org/10.1007/978-0-387-30162-4_412

ripgrep – Text-search tool that searches files and directories for matching patterns.
https://github.com/BurntSushi/ripgrep